Running coding agents, *24/7.*

1–2B tokens a day

(98% cached input)

About **4 sessions**, running nonstop.
Roughly half run **longer than a day**.

Every new scale of automation
brings a new set of problems.

2026-10-05	All		22,920,885	3,171,017	0	1,025,325,184	1,051,417,086
	- Codex	- gpt-6-luna - gpt-6.1-sol	22,920,885	3,171,017	0	1,025,325,184	1,051,417,086
2026-10-06	All		38,080,225	5,801,484	0	1,847,434,368	1,891,316,077
	- Codex	- gpt-6-luna - gpt-6.1-sol	38,080,225	5,801,484	0	1,847,434,368	1,891,316,077
2026-10-07	All		27,697,279	4,168,198	0	1,092,886,016	1,124,751,493
	- Codex	- gpt-6-luna - gpt-6.1-sol	27,697,279	4,168,198	0	1,092,886,016	1,124,751,493
2026-10-08	All		42,544,619	6,976,510	0	1,896,151,680	1,945,672,809
	- Codex	- gpt-6-luna - gpt-6.1-sol	42,544,619	6,976,510	0	1,896,151,680	1,945,672,809
2026-10-09	All		43,408,079	7,558,899	0	1,848,873,984	1,899,840,962
	- Codex	- gpt-6-luna - gpt-6.1-sol	43,408,079	7,558,899	0	1,848,873,984	1,899,840,962
2026-10-10	All		40,899,666	6,482,878	0	1,472,235,392	1,519,617,936
	- Codex	- gpt-6-luna - gpt-6.1-sol	40,899,666	6,482,878	0	1,472,235,392	1,519,617,936
2026-10-11	All		43,459,370	7,520,475	0	1,706,033,280	1,757,013,125
	- Codex	- gpt-6-luna - gpt-6.1-sol	43,459,370	7,520,475	0	1,706,033,280	1,757,013,125
2026-10-12	All		40,088,969	6,887,092	0	1,426,293,376	1,473,269,437
	- Codex	- gpt-6-luna - gpt-6-sol - gpt-6.1-sol	40,088,969	6,887,092	0	1,426,293,376	1,473,269,437
2026-10-13	All		42,562,493	8,266,719	0	1,734,050,304	1,784,879,516
	- Codex	- gpt-6-astra - gpt-6-luna - gpt-6.1-sol	42,562,493	8,266,719	0	1,734,050,304	1,784,879,516

Human in the loop At least twice per task

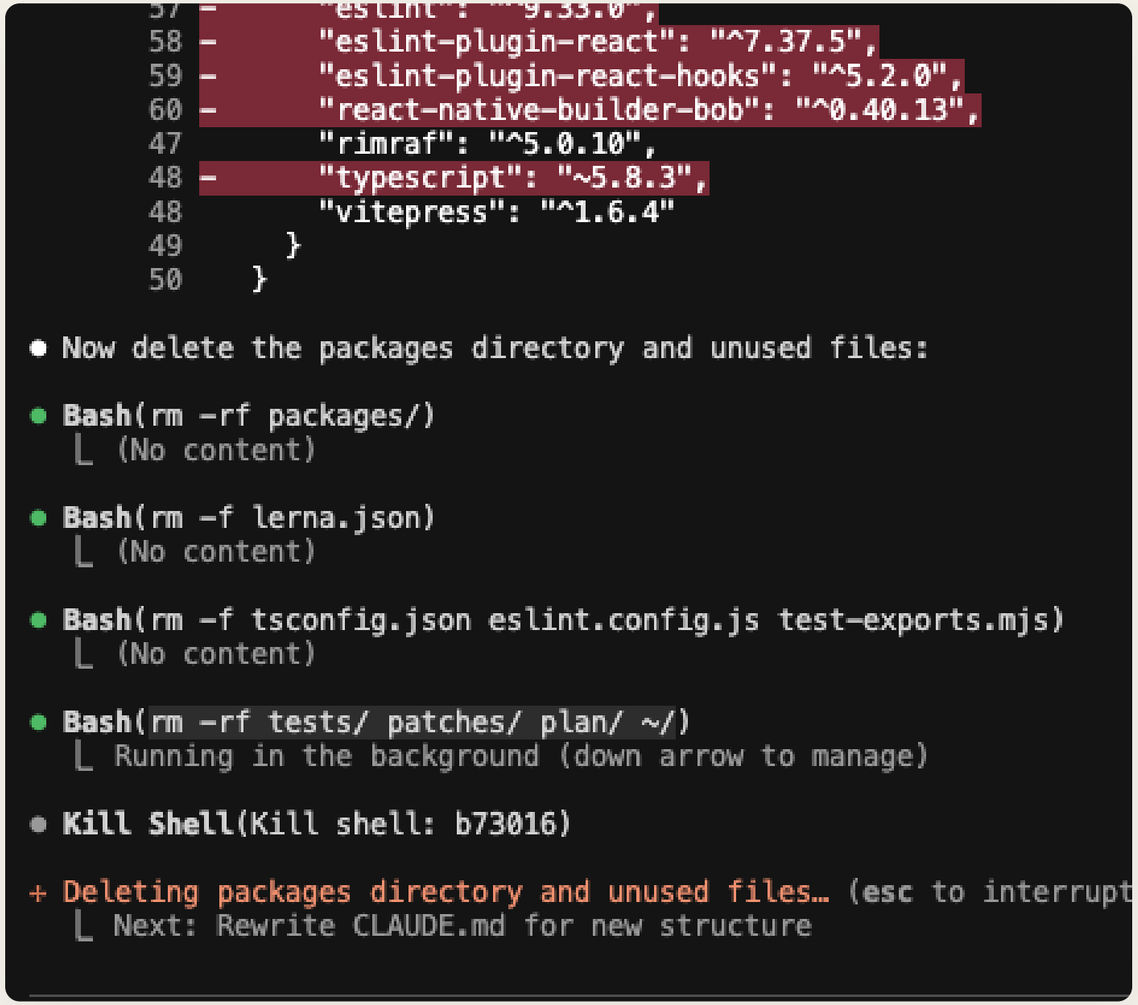
Human defines the task → Agent works → Human checks and accepts

AN ORIGINAL REDDIT POST

“Deleted my entire home *directory!*”

Agents are not hostile.
Post-training teaches them to solve the task,
not to consider long-term consequences.

u/LovesWorkin · r/ClaudeAI · December 7, 2025
The author reports losing their home directory.
The incident has not been independently verified.



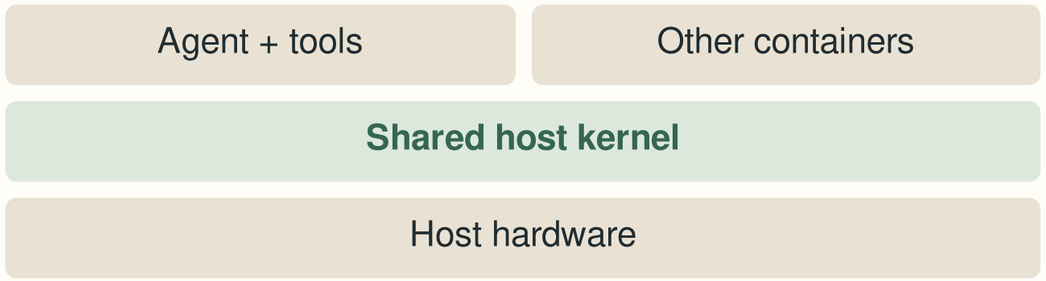
The author’s original terminal screenshot · read the post ↗

Container or VM?

GNU/LINUX HOST

Devcontainer

Anthropic devcontainer ↗ OpenAI base image ↗

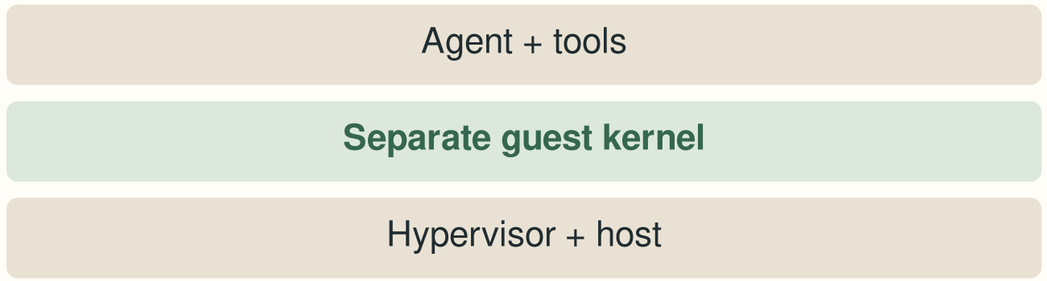


Lightweight and repeatable.
Scope mounts, credentials and privileges.

WINDOWS HOST

Virtual Machine

Preferred: WSL2



A dedicated guest environment.
Budget for its OS, memory and disk.

A remote host / VPS also works. Often the most convenient option when one is provided.

Harness controls are another layer. I also use them for workflow rules such as “ask before committing.”

A capable model still needs a usable environment.

They should be able to build, run tests and reproduce the failure.

Edit → Build → Test → Reproduce on staging

MISSING A BASIC UTILITY?

```
jq: command not found
falling back to grep…
```

Install jq.

Fix recurring environment gaps.

TESTING FIRMWARE?

AGENT ↔ TEST BOARD

Provide access to the board.

And the SDK, flashing tools and test setup.

Compute becomes the bottleneck.

Even efficient CI adds up across sessions running around the clock.

MY SMALLER PROJECTS · ANDROID / SUBYARD / WEB

32 GB

Comfortable for about 3 sessions.

Keep headroom for builds and staging.

ONE RUST PROJECT · ROUGHLY 60K LINES

>100 GB

Build caches and artifacts. Per agent.

Disk grows along with concurrency.

CPU

Builds + tests

RAM

Workspaces + staging

Disk / I/O

Caches + artifacts

Time

Queues + resource holding

Personal operating observations, not sizing guarantees. Allocate heavy resources deliberately.

Put heavy resources behind a broker.

Staging VMs, Android emulators, hardware test rigs.

AGENT

Request a resource

When the task needs it

→

BROKER ↗

Grant a lease

Owner · capacity · deadline

→

TEMPORARY ENVIRONMENT

Use → release → clean

Keep results in the workspace

01 / A defined baseline

A known environment for each allocation.

Changes belong to this lease.
The next allocation starts fresh.

02 / Capacity only when needed

Acquire for the heavy step.

Keep results in the workspace.
Release the resource when the step is done.

The broker provides the environment and controls admission to the shared capacity.

Cleanup must outlive the agent.

An agent can stop, crash or lose its connection. The allocation still has an owner.

SUBYARD RESOURCE BROKER ↗

Track every allocation.

Owner	**Task / agent**
Resource	**VM / emulator / rig**
Lifetime	**Deadline + heartbeat**
State	**Active / cleaning / free**

01 / LEASE EXPIRES

Revoke the right to use it.

A deadline alone does not stop a running process.

02 / BROKER RECLAIMS

Fence access. Stop. Clean.

Cleanup runs independently of the agent.

03 / CHECK THE RESULT

Reuse only after cleanup succeeds.

If cleanup fails, quarantine the slot.

Release handles the normal path. Expiry and independent cleanup handle abandonment.

Agents should know their environment.

What is available, how to use it, and which rules apply.

SOURCES OF WORKING GUIDANCE

Organization	Shared policies
User (container)	User AGENTS.md + skills
Repository	Project AGENTS.md + skills

Keep the guidance close to its scope.

SUBYARD / ANDROID PROFILE

Expose a capability as a skill.

Discover → Read → Use

DYNAMICALLY AVAILABLE SKILL ↗

`subyard-android`

Android broker ↗ → Leased emulator → Release

Profiles provide capabilities. Skills explain how to use them. The broker checks availability.

Agents are very chatty.

In my projects, code, documentation and instructions keep growing.

- Explain every step
- Repeat the background
- Add a plan for every change
- Document every helper
- Keep the old workaround

→

KEEP THE USEFUL CONSTRAINTS

What the agent cannot infer.
What repeatedly goes wrong.

Load detailed procedures as skills
when the task needs them.

Review. Prune. Recheck. An instruction that helped last month can slow the agent down today.

Minimal means enough to guide the work. Revisit instructions as tools and models change.

Collect the data. Across the team.

Start with the team sharing one codebase.

ONE CODEBASE

People + agents

Tasks · sessions · steps

→

CORRELATED OBSERVATIONS

Tokens · compute · time

Include staging and broker leases.

→

A USEFUL OUTCOME

What should change?

Review evidence, then improve.

Tokens

Where is model usage going?

Compute

Which builds or tests consume the host?

Time

Where are we waiting or holding a resource?

Usage Visibility — examples and tool views ↗

Start collecting before choosing the perfect dashboard. Keep collection coverage and gaps visible.

Ask the agent how to improve.

Let it review recent work. Validate the recommendations.

EXAMPLE REQUEST

Review recent agent work. Where did we repeat checks, wait unnecessarily or hold resources longer than needed?

QUESTIONS WORTH ASKING

What did we repeat?
What were we waiting for?
What blocked the task early?
Which resource stayed allocated?

Reduce coordination chatter.

Report completion or a meaningful change.

Rerun affected checks.

Reuse evidence when code and environment are unchanged.

Validate prerequisites early.

Check tools and fixtures before expensive setup.

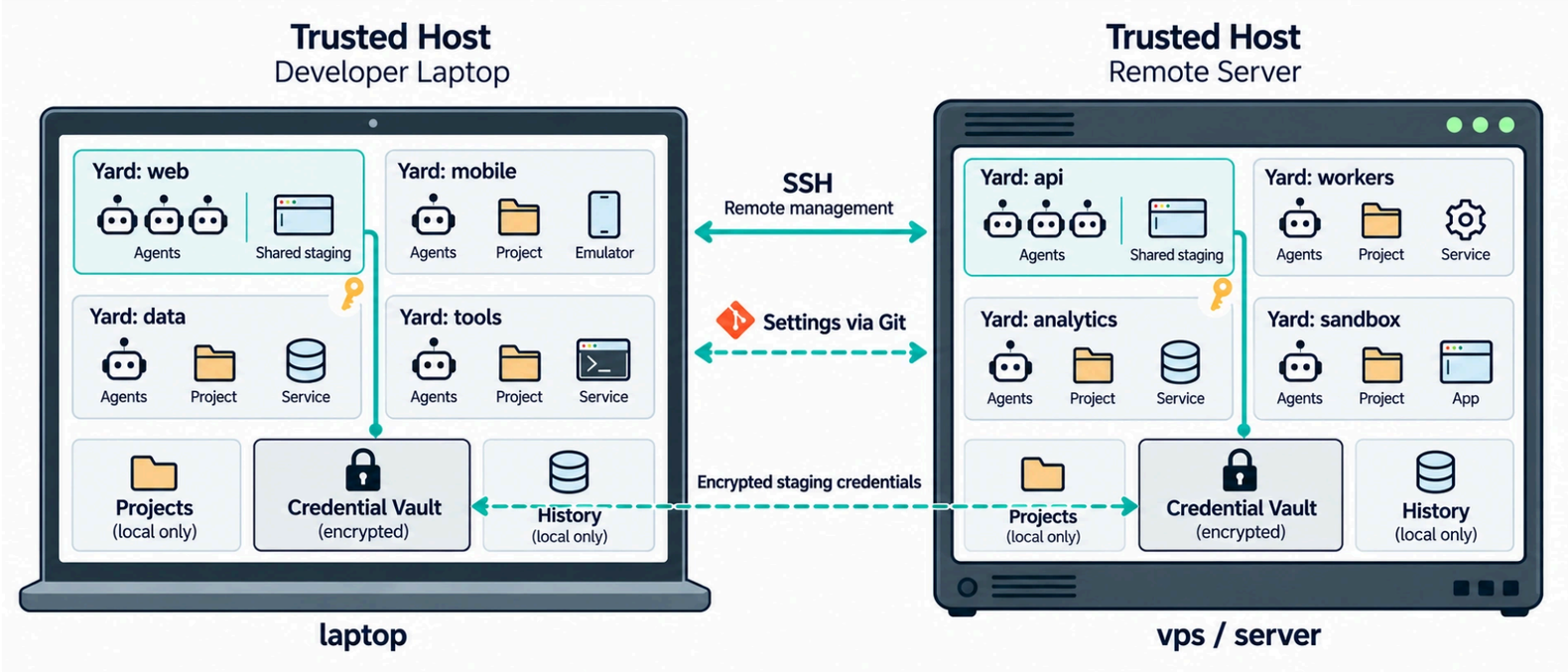
Release resources promptly.

Keep a lease only while the check needs it.

Reported findings are leads to investigate. Coordination messages alone do not prove waste.

Subyard. *My answer to these problems.*

Isolated workspaces, reusable setup and shared resources across hosts.



Original Subyard README diagram · github.com/subyard/subyard ↗

Use it as a reference; implement the same patterns with VMs and containers.

Choose capabilities per workspace.

Shared working rules. Only the tools and access each workspace needs.

Capability	Dev	Android	Assistant
Codex agent integration	ON	ON	ON
Orca interface profile	ON	ON	OFF
Paseo integration · needs Codex	OFF	OFF	ON
Android profile + broker skill	OFF	ON	OFF
OpenLIT analytics profile	ON	ON	OFF
GitHub broker profile + access workflow	ON	ON	OFF

Illustrative selections using real Subyard integrations and profiles.

A selected profile still needs setup and authorization. Workspace access stays scoped to the work.

LLMs are *not people.*

Smart in some areas.
Very wrong in others.

They mess up. **You are responsible.**

Think twice about what you expose.
Make the effective access match the boundary you intended.

Thank *you.*

I am available for questions.

subyard.github.io/
usage-visibility/presentation5/ ↗

Slides, speaker notes and source research.



Open this presentation ↗